CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 3

Name: Prionti Das | **UID: 24BAI71005**
Branch: CSE-AIML(CS221) | **Section/Group: 24AIT_NTPP3[B]**
Semester:5th | **Date of Performance: 22/07/26**
Subject: Full Stack -II | **Subject Code: 24CSP-337**

EXPERIMENT – 3.1

1. AIM:

To design and implement a secure authentication system using JWT for user login and session management.

2. OBJECTIVE:

1. To understand authentication mechanisms in web applications
2. To implement token-based authentication using JWT
3. To manage user sessions in a stateless architecture
4. To handle token storage and validation securely

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Node.js
3. JWT libraries
4. Browser

4. THEORY:

Authentication is the process of verifying the identity of a user before granting access to system resources. Traditional session-based authentication relies on server-side session storage, which can become inefficient in distributed systems. JWT-based authentication introduces a stateless approach, where the server issues a signed token containing user information. This token is stored on the client and sent with each request for verification.

A JWT consists of three parts:

- Header – Defines the algorithm
- Payload – Contains user data (claims)
- Signature – Ensures integrity and authenticity

5. IMPLEMENTATION/PROCEDURE:

1. Create login form UI
2. Validate user credentials *(mock or static)*
3. Generate or simulate JWT token
4. Store token (localStorage/sessionStorage)
5. Attach token to requests
6. Decode token to extract user information

6. CODE :

index . js :

```
`import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

index.css :

```
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

code {
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',
    monospace;
}
```

app . js :

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './components/Login';
import Dashboard from './components/Dashboard';
import Navbar from './components/Navbar';
import ProtectedRoute from './components/ProtectedRoute';
import './App.css';

function App() {
  return (
    <Router>
      <div className="App">
        <Navbar />
        <Routes>
          <Route path="/login" element={<Login />} />
          <Route
            path="/dashboard"
            element={
              <ProtectedRoute>
                <Dashboard />
              </ProtectedRoute>
            }
          />
          <Route path="/" element={<Navigate to="/login" />} />
```

```
        </Routes>
      </div>
    </Router>
  );
}
export default App;
```

app.css :

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Arial, sans-serif;
  background: #f7fafc;
  color: #2d3748;
}

.App {
  min-height: 100vh;
}

/* Smooth transitions */
button {
  transition: all 0.3s ease;
}

input {
  transition: all 0.3s ease;
}

/* Scrollbar styling */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

::-webkit-scrollbar-track {
  background: #f1f1f1;
  border-radius: 10px;
}

::-webkit-scrollbar-thumb {
  background: #888;
  border-radius: 10px;
}

::-webkit-scrollbar-thumb:hover {
  background: #555;
}
```

auth . js :

```
// JWT token management functions
```

```
// Store token in localStorage
export const setToken = (token) => {
  localStorage.setItem('authToken', token);
};

// Retrieve token from localStorage
export const getToken = () => {
  return localStorage.getItem('authToken');
};

// Remove token from localStorage
export const removeToken = () => {
  localStorage.removeItem('authToken');
};

// Check if user is authenticated
export const isAuthenticated = () => {
  const token = getToken();
  if (!token) return false;

  try {
    // Decode token to check expiration
    const payload = JSON.parse(atob(token.split('.')[1]));
    const expirationTime = payload.exp * 1000;
    return Date.now() < expirationTime;
  } catch (error) {
    return false;
  }
};

// Decode JWT token to get user info
export const decodeToken = () => {
  const token = getToken();
  if (!token) return null;

  try {
    const payload = JSON.parse(atob(token.split('.')[1]));
    return payload;
  } catch (error) {
    return null;
  }
};

// Simulate JWT token generation (for demo)
export const generateMockToken = (userData) => {
  // In real scenario, this would be done server-side
  const header = btoa(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));
  const payload = btoa(
    JSON.stringify({
      ...userData,
      exp: Math.floor(Date.now() / 1000) + 3600 // 1 hour expiry
    })
  );
  const signature = btoa('mock-signature-' + Date.now());

  return `${header}.${payload}.${signature}`;
```

```
};
```

api . js :

```
import { getToken } from '../utils/auth';

// Simulated API service with token handling
export const apiService = {
  // GET request simulation
  get: (endpoint) => {
    const token = getToken();

    console.log(`[API] GET request to ${endpoint}`);
    console.log(
      `[API] Authorization: Bearer ${
        token ? token.substring(0, 30) + '...' : 'No token'
      }`
    );

    // Simulate API response
    return new Promise((resolve) => {
      setTimeout(() => {
        resolve({
          data: {
            message: `Data from ${endpoint}`,
            timestamp: new Date().toISOString(),
            authenticated: !!token,
          },
        });
      }, 500);
    });
  },

  // POST request simulation
  post: (endpoint, data) => {
    const token = getToken();

    console.log(`[API] POST request to ${endpoint}`);
    console.log(`[API] Data:`, data);
    console.log(
      `[API] Authorization: Bearer ${
        token ? token.substring(0, 30) + '...' : 'No token'
      }`
    );

    return new Promise((resolve) => {
      setTimeout(() => {
        resolve({
          data: {
            message: `Data posted to ${endpoint}`,
            received: data,
            timestamp: new Date().toISOString(),
          },
        });
      }, 500);
    });
  },
```

```
};
```

Navbar . js :

```
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { isAuthenticated, removeToken } from '../utils/auth';
import './Navbar.css';

const Navbar = () => {
  const navigate = useNavigate();
  const authenticated = isAuthenticated();

  const handleLogout = () => {
    removeToken();
    navigate('/login');
  };

  return (
    <nav className="navbar">
      <div className="navbar-brand">
        <Link to="/">JWT Auth Demo</Link>
      </div>

      <div className="navbar-links">
        {authenticated ? (
          <>
            <Link to="/dashboard">Dashboard</Link>
            <button onClick={handleLogout} className="nav-logout-btn">
              Logout
            </button>
          </>
        ) : (
          <>
            <Link to="/login">Login</Link>
          </>
        )}
      </div>
    </nav>
  );
};

export default Navbar;
```

Navbar . css :

```
.navbar {
  background: #2d3748;
  padding: 15px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: white;
}

.navbar-brand a {
  color: white;
  text-decoration: none;
```

```
  font-size: 20px;
  font-weight: bold;
}

.navbar-brand a:hover {
  color: #a0aec0;
}

.navbar-links {
  display: flex;
  gap: 20px;
  align-items: center;
}

.navbar-links a {
  color: white;
  text-decoration: none;
  padding: 8px 12px;
  border-radius: 5px;
  transition: background 0.3s;
}

.navbar-links a:hover {
  background: #4a5568;
}

.nav-logout-btn {
  background: #e53e3e;
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: 5px;
  cursor: pointer;
  transition: background 0.3s;
}

.nav-logout-btn:hover {
  background: #c53030;
}
```

Login . js :

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { setToken, generateMockToken, decodeToken } from '../utils/auth';
import './Login.css';

const Login = () => {
  const [credentials, setCredentials] = useState({
    email: '',
    password: ''
  });

  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
```

```
// Mock user database (in real app, this would be a server call)
const mockUsers = [
  {
    email: 'user_24bai71005@cuchd.in',
    password: 'user_24BAI71005',
    name: 'Prionti Das',
    role: 'user'
  },
  {
    email: 'admin_24bai71005@cuchd.in',
    password: 'admin24BAI71005',
    name: 'Prionti Das',
    role: 'admin'
  }
];

const handleChange = (e) => {
  setCredentials({
    ...credentials,
    [e.target.name]: e.target.value
  });
};

const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setLoading(true);

  try {
    // Validate credentials (mock validation)
    const user = mockUsers.find(
      (u) =>
        u.email === credentials.email &&
        u.password === credentials.password
    );

    if (!user) {
      throw new Error('Invalid credentials');
    }

    // Generate JWT token (mock)
    const tokenData = {
      id: Date.now(),
      email: user.email,
      name: user.name,
      role: user.role
    };

    // In real application, token would come from server
    const token = generateMockToken(tokenData);

    // Store token
    setToken(token);

    // Decode and display token info (for demonstration)
    const decoded = decodeToken();
```

```
      console.log('Decoded Token:', decoded);
      console.log('Full Token:', token);

      // Redirect to dashboard
      navigate('/dashboard');

    } catch (err) {
      setError(err.message || 'Login failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="login-container">
      <div className="login-card">
        <h2>Login</h2>
        <h3>JWT Authentication Demo</h3>

        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label>Email</label>
            <input
              type="email"
              name="email"
              placeholder="Enter your email"
              value={credentials.email}
              onChange={handleChange}
              required
            />
          </div>

          <div className="form-group">
            <label>Password</label>
            <input
              type="password"
              name="password"
              placeholder="Enter your password"
              value={credentials.password}
              onChange={handleChange}
              required
            />
          </div>

          {error && <div className="error-message">{error}</div>}

          <button
            type="submit"
            className="login-btn"
            disabled={loading}
          >
            {loading ? 'Logging in...' : 'Login'}
          </button>
        </form>

        <div className="demo-credentials">
```

```
            <h4>Demo Credentials:</h4>
            <p>User: user_24bai71005@cuchd.in / user_24BAI71005</p>
            <p>Admin: admin_24bai71005@cuchd.in / admin24BAI71005</p>
          </div>
        </div>
      </div>
    );
};

export default Login;
```

Login . css:

```
.login-container {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.login-card {
  background: white;
  padding: 40px;
  border-radius: 10px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
  width: 100%;
  max-width: 400px;
}

.login-card h2 {
  margin-bottom: 5px;
  color: #333;
}

.login-card h3 {
  margin-top: 0;
  margin-bottom: 30px;
  color: #666;
  font-weight: normal;
  font-size: 16px;
}

.form-group {
  margin-bottom: 20px;
}

.form-group label {
  display: block;
  margin-bottom: 5px;
  color: #555;
  font-weight: 500;
}

.form-group input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ddd;
```

```
  border-radius: 5px;
  font-size: 16px;
  box-sizing: border-box;
 }

 .form-group input:focus {
  outline: none;
  border-color: #667eea;
 }

 .login-btn {
  width: 100%;
  padding: 12px;
  background: #667eea;
  color: white;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.3s;
 }

 .login-btn:hover {
  background: #5a67d8;
 }

 .login-btn:disabled {
  opacity: 0.7;
  cursor: not-allowed;
 }

 .error-message {
  color: #e53e3e;
  background: #fff5f5;
  padding: 10px;
  border-radius: 5px;
  margin-bottom: 15px;
  border: 1px solid #fc8181;
 }

 .demo-credentials {
  margin-top: 20px;
  padding: 15px;
  background: #f7fafc;
  border-radius: 5px;
  border: 1px solid #e2e8f0;
 }

 .demo-credentials h4 {
  margin: 0 0 10px 0;
  color: #4a5568;
 }

 .demo-credentials p {
  margin: 5px 0;
```

```
  color: #718096;
  font-size: 14px;
}
```

Dashboard . js :

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { decodeToken, removeToken } from '../utils/auth';
import './Dashboard.css';

const Dashboard = () => {
  const [userData, setUserData] = useState(null);
  const [tokenInfo, setTokenInfo] = useState(null);
  const navigate = useNavigate();

  useEffect(() => {
    // Decode token to get user information
    const decoded = decodeToken();

    if (decoded) {
      setUserData(decoded);

      // Display token information for demonstration
      const token = localStorage.getItem('authToken');

      if (token) {
        const parts = token.split('.');

        setTokenInfo({
          header: JSON.parse(atob(parts[0])),
          payload: decoded,
          signature: parts[2]
            ? parts[2].substring(0, 20) + '...'
            : 'N/A'
        });
      }
    }
  }, []);

  const handleLogout = () => {
    removeToken();
    navigate('/login');
  };

  if (!userData) {
    return <div className="loading">Loading...</div>;
  }

  return (
    <div className="dashboard-container">
      <div className="dashboard-header">
        <h1>Welcome, {userData.name}!</h1>

        <button onClick={handleLogout} className="logout-btn">
          Logout
        </button>
      </div>
```

```
<div className="dashboard-content">
  <div className="user-info">
    <h2>User Information</h2>

    <div className="info-card">
      <p>
        <strong>Name:</strong> {userData.name}
      </p>

      <p>
        <strong>Email:</strong> {userData.email}
      </p>

      <p>
        <strong>Role:</strong> {userData.role}
      </p>

      <p>
        <strong>User ID:</strong> {userData.id}
      </p>
    </div>
  </div>

  <div className="token-info">
    <h2>JWT Token Information</h2>

    <div className="info-card">
      <h4>Token Structure:</h4>

      <div className="token-parts">
        <div className="token-part">
          <h5>Header</h5>
          <pre>{JSON.stringify(tokenInfo?.header, null, 2)}</pre>
        </div>

        <div className="token-part">
          <h5>Payload</h5>
          <pre>{JSON.stringify(tokenInfo?.payload, null, 2)}</pre>
        </div>

        <div className="token-part">
          <h5>Signature (partial)</h5>
          <pre>{tokenInfo?.signature || 'N/A'}</pre>
        </div>
      </div>

      <div className="full-token">
        <h5>Full Token:</h5>
        <p className="token-string">
          {localStorage.getItem('authToken')}
        </p>
      </div>
    </div>
  </div>
```

```
        <div className="requests-info">
          <h2>Stateless Authentication Flow</h2>

          <div className="flow-diagram">
            <div className="flow-step">
              <span className="step-number">1</span>
              <p>Token stored in localStorage</p>
            </div>

            <div className="flow-arrow">→</div>

            <div className="flow-step">
              <span className="step-number">2</span>
              <p>Token sent with each request</p>
            </div>

            <div className="flow-arrow">→</div>

            <div className="flow-step">
              <span className="step-number">3</span>
              <p>Server validates token</p>
            </div>

            <div className="flow-arrow">→</div>

            <div className="flow-step">
              <span className="step-number">4</span>
              <p>User authenticated</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};

export default Dashboard;
```

Dashboard . css :

```
.dashboard-container {
  min-height: 100vh;
  background: #f7fafc;
  padding: 20px;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: white;
  padding: 20px 30px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  margin-bottom: 30px;
}
```

```
.dashboard-header h1 {
  margin: 0;
  color: #2d3748;
}

.logout-btn {
  padding: 10px 20px;
  background: #e53e3e;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-weight: 600;
  transition: background 0.3s;
}

.logout-btn:hover {
  background: #c53030;
}

.dashboard-content {
  max-width: 1200px;
  margin: 0 auto;
}

.dashboard-content h2 {
  color: #2d3748;
  margin-bottom: 20px;
}

.info-card {
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  margin-bottom: 30px;
}

.info-card p {
  margin: 10px 0;
  color: #4a5568;
}

.token-parts {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
  margin: 20px 0;
}

.token-part {
  background: #f7fafc;
  padding: 15px;
  border-radius: 5px;
  border: 1px solid #e2e8f0;
}
```

```
.token-part h5 {
  margin: 0 0 10px 0;
  color: #4a5568;
}

.token-part pre {
  background: #2d3748;
  color: #e2e8f0;
  padding: 10px;
  border-radius: 5px;
  overflow-x: auto;
  font-size: 12px;
  margin: 0;
}

.full-token {
  margin-top: 20px;
  padding-top: 20px;
  border-top: 1px solid #e2e8f0;
}

.token-string {
  background: #f7fafc;
  padding: 10px;
  border-radius: 5px;
  word-break: break-all;
  font-family: monospace;
  font-size: 12px;
  border: 1px solid #e2e8f0;
}

.flow-diagram {
  display: flex;
  align-items: center;
  justify-content: space-around;
  background: white;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  flex-wrap: wrap;
}

.flow-step {
  text-align: center;
  flex: 1;
  min-width: 120px;
}

.step-number {
  display: inline-block;
  background: #667eea;
  color: white;
  width: 30px;
  height: 30px;
  line-height: 30px;
```

```
  border-radius: 50%;
  font-weight: bold;
  margin-bottom: 10px;
}

.flow-step p {
  margin: 0;
  color: #4a5568;
  font-weight: 500;
}

.flow-arrow {
  font-size: 24px;
  color: #a0aec0;
  padding: 0 10px;
}

.loading {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  font-size: 18px;
  color: #4a5568;
}

@media (max-width: 768px) {
  .token-parts {
    grid-template-columns: 1fr;
  }

  .flow-diagram {
    flex-direction: column;
  }

  .flow-arrow {
    transform: rotate(90deg);
  }
}
```

ProtectedRoute . js :

```
import React from 'react';
import { Navigate } from 'react-router-dom';
import { isAuthenticated } from '../utils/auth';

const ProtectedRoute = ({ children }) => {
  if (!isAuthenticated()) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

package . json :

```
{
 "name": "jwt-auth-app",
 "version": "0.1.0",
 "private": true,
 "dependencies": {
  "@testing-library/dom": "^10.4.1",
  "@testing-library/jest-dom": "^6.9.1",
  "@testing-library/react": "^16.3.2",
  "@testing-library/user-event": "^13.5.0",
  "axios": "^1.19.0",
  "bcryptjs": "^3.0.3",
  "jsonwebtoken": "^9.0.3",
  "react": "^19.2.8",
  "react-dom": "^19.2.8",
  "react-router-dom": "^7.18.2",
  "react-scripts": "5.0.1",
  "web-vitals": "^2.1.4"
 },
 "scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
 },
 "eslintConfig": {
  "extends": [
   "react-app",
   "react-app/jest"
  ]
 },
 "browserslist": {
  "production": [
   ">0.2%",
   "not dead",
   "not op_mini all"
  ],
  "development": [
   "last 1 chrome version",
   "last 1 firefox version",
   "last 1 safari version"
  ]
 },
 "devDependencies": {
  "@types/jsonwebtoken": "^9.0.10"
 }
}
```

7. OUTPUT:

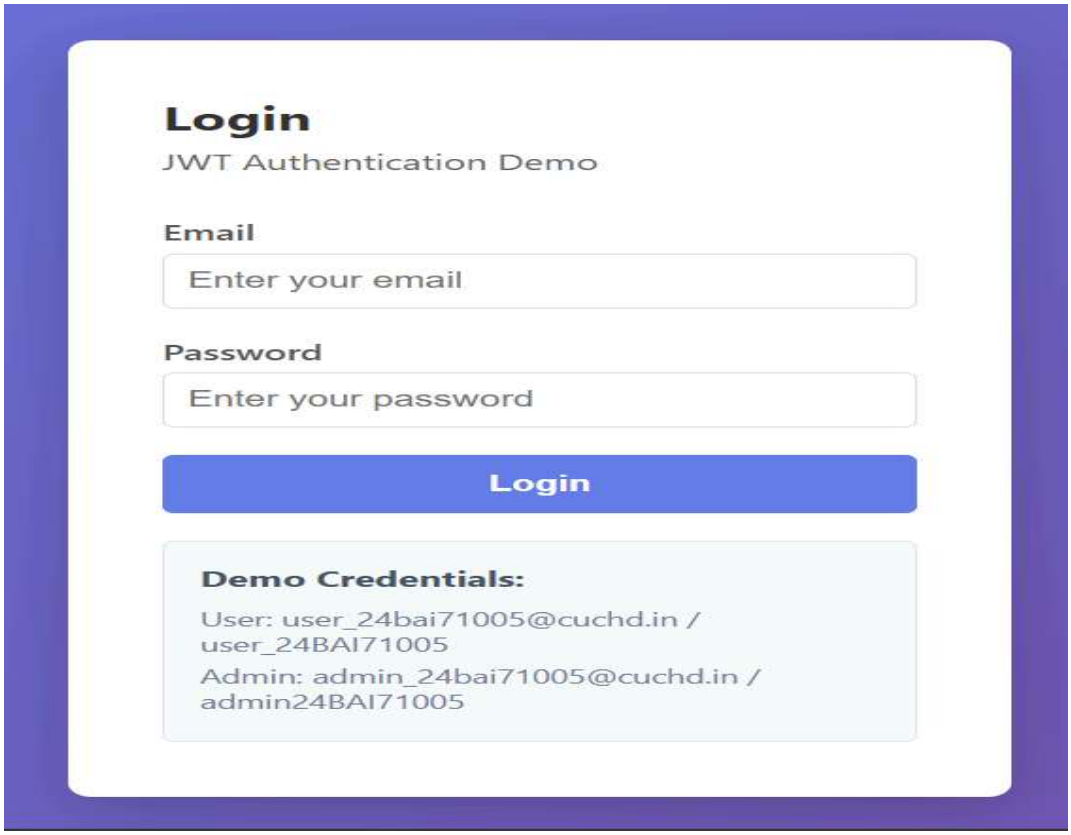


Fig 3.1 : JWT Authentication Login Page

This experiment demonstrates the implementation of **JWT (JSON Web Token)** authentication in a React application. A user logs in using valid credentials, after which a JWT token is generated and stored in the browser's localStorage. Protected routes ensure that only authenticated users can access the dashboard. The application also decodes the token to display user information and token details, illustrating how JWT enables secure, stateless authentication without maintaining server-side session data.

8. LEARNING OUTCOME:

- Understand the concept of authentication in web applications.
- Implement JWT-based authentication for secure user login.
- Generate and manage JWT tokens for stateless session handling.
- Store authentication tokens securely using localStorage.
- Decode JWT tokens to retrieve user information.
- Implement login, logout, and session management functionality.
- Develop secure authentication workflows in React applications.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 3.2

1. AIM:

To implement role-based access control and secure application routes based on user permissions.

2. OBJECTIVE:

1. To understand authorization mechanisms in applications
2. To implement RBAC for access control
3. To protect routes based on user roles
4. To dynamically render UI elements based on permissions

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Node.js
3. React Router
4. State management

4. THEORY:

While authentication verifies user identity, authorization determines what actions a user is permitted to perform. Role-Based Access Control (RBAC) is a widely used model where users are assigned roles, and each role has defined permissions.

Common roles include:

- Admin
- Editor
- Viewer

RBAC ensures:

- Controlled access to sensitive features
- Simplified permission management
- Improved system security

Route protection is implemented to restrict navigation to certain pages unless the user has appropriate permissions.

5. IMPLEMENTATION/PROCEDURE :

1. Define user roles (Admin, Editor, Viewer)
2. Store role in authentication state
3. Implement protected routes using React Router
4. Restrict access based on role
5. Conditionally render UI elements (buttons, pages)
6. Redirect unauthorized users

6. CODE :

authService . js :

```
const users = [
  {
    username: "admin_24bai71005@cuchd.in",
    password: "admin_24BAI71005",
    role: "Admin",
```

```
  },
  {
    username: "editor_24bai71005@cuchd.in",
    password: "editor_24BAI71005",
    role: "Editor",
  },
  {
    username: "viewer_24bai71005@cuchd.in",
    password: "viewer_24BAI71005",
    role: "Viewer",
  },
];

export const authenticateUser = (username, password) => {
  const user = users.find(
    (u) => u.username === username && u.password === password
  );

  if (!user) return null;

  // Create a simple JWT-like token
  const header = btoa(
    JSON.stringify({
      alg: "HS256",
      typ: "JWT",
    })
  );

  const payload = btoa(
    JSON.stringify({
      username: user.username,
      role: user.role,
      exp: Date.now() + 3600000, // 1 hour expiry
    })
  );

  const signature = btoa("rbac-secret-key");

  return `${header}.${payload}.${signature}`;
};

export const getUsers = () => users;
```

token . js :

```
export const getUser = () => {
  const token = localStorage.getItem("token");

  if (!token) return null;

  try {
    const payload = JSON.parse(atob(token.split(".")[1]));

    // Check if token has expired
    if (payload.exp && payload.exp < Date.now()) {
      localStorage.removeItem("token");
      return null;
    }
```

```
    return payload;
  } catch (error) {
    localStorage.removeItem("token");
    return null;
  }
};

export const getRole = () => {
  const user = getUser();
  return user?.role;
};

export const getUsername = () => {
  const user = getUser();
  return user?.username;
};

export const isAuthenticated = () => {
  return getUser() !== null;
};

export const logout = () => {
  localStorage.removeItem("token");
};
```

ProtectedRoute . js :

```
import { Navigate } from "react-router-dom";
import { isAuthenticated, getRole } from "../utils/token";

function ProtectedRoute({ children, allowedRoles }) {
  if (!isAuthenticated()) {
    return <Navigate to="/" replace />;
  }

  const role = getRole();

  if (!allowedRoles.includes(role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return children;
}

export default ProtectedRoute;
```

Navbar . js :

```
import { useNavigate } from "react-router-dom";
import { getUsername, getRole, logout } from "../utils/token";

function Navbar() {
  const navigate = useNavigate();
  const username = getUsername();
  const role = getRole();

  const handleLogout = () => {
    logout();
    navigate("/");
```

```
  };

  return (
    <nav
      style={{
        backgroundColor: "#333",
        color: "white",
        padding: "15px 20px",
        display: "flex",
        justifyContent: "space-between",
        alignItems: "center",
        marginBottom: "20px",
      }}
    >
      <div>
        <h2 style={{ margin: 0 }}>RBAC Demo App</h2>
      </div>

      <div
        style={{
          display: "flex",
          alignItems: "center",
          gap: "20px",
        }}
      >
        <span>Welcome, {username}</span>

        <span
          style={{
            backgroundColor: "#007bff",
            padding: "5px 10px",
            borderRadius: "5px",
          }}
        >
          Role: {role}
        </span>

        <button
          onClick={handleLogout}
          style={{
            padding: "8px 15px",
            backgroundColor: "#dc3545",
            color: "white",
            border: "none",
            borderRadius: "5px",
            cursor: "pointer",
          }}
        >
          Logout
        </button>
      </div>
    </nav>
  );
}

export default Navbar;
```

unauthorized . js :

```
import { useNavigate } from "react-router-dom";

function Unauthorized() {
  const navigate = useNavigate();

  return (
    <div
      style={{
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        justifyContent: "center",
        height: "60vh",
      }}
    >
      <h1
        style={{
          color: "#dc3545",
          fontSize: "48px",
        }}
      >
        403
      </h1>

      <h2>Access Denied</h2>

      <p>You do not have permission to access this page.</p>

      <button
        onClick={() => navigate("/")}
        style={{
          padding: "10px 20px",
          backgroundColor: "#007bff",
          color: "white",
          border: "none",
          borderRadius: "5px",
          cursor: "pointer",
          marginTop: "20px",
        }}
      >
        Back to Login
      </button>
    </div>
  );
}

export default Unauthorized;
```

Login . js :

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { authenticateUser } from "../services/authService";

function Login() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
```

```
  const [error, setError] = useState("");

  const navigate = useNavigate();

  const handleLogin = (e) => {
    e.preventDefault();
    setError("");

    const token = authenticateUser(username, password);

    if (!token) {
      setError("Invalid Username or Password");
      return;
    }

    localStorage.setItem("token", token);

    // Decode token to get role
    const payload = JSON.parse(atob(token.split(".")[1]));

    // Redirect based on role
    switch (payload.role) {
      case "Admin":
        navigate("/admin");
        break;

      case "Editor":
        navigate("/editor");
        break;

      case "Viewer":
        navigate("/viewer");
        break;

      default:
        navigate("/");
    }
  };

  return (
    <div
      style={{
        display: "flex",
        justifyContent: "center",
        alignItems: "center",
        height: "100vh",
        backgroundColor: "#f5f5f5",
      }}
    >
      <div
        style={{
          backgroundColor: "white",
          padding: "40px",
          borderRadius: "10px",
          boxShadow: "0 0 10px rgba(0,0,0,0.1)",
          width: "350px",
```

```
        }}
      >
        <h2 style={{ textAlign: "center", marginBottom: "30px" }}>
          RBAC Login
        </h2>

        {error && (
          <div
            style={{
              color: "#dc3545",
              backgroundColor: "#f8d7da",
              padding: "10px",
              borderRadius: "5px",
              marginBottom: "20px",
              textAlign: "center",
            }}
          >
            {error}
          </div>
        )}

        <form onSubmit={handleLogin}>
          <div style={{ marginBottom: "20px" }}>
            <label
              style={{
                display: "block",
                marginBottom: "5px",
              }}
            >
              Username
            </label>

            <input
              type="text"
              value={username}
              onChange={(e) => setUsername(e.target.value)}
              style={{
                width: "100%",
                padding: "10px",
                border: "1px solid #ddd",
                borderRadius: "5px",
                boxSizing: "border-box",
              }}
              required
            />
          </div>

          <div style={{ marginBottom: "20px" }}>
            <label
              style={{
                display: "block",
                marginBottom: "5px",
              }}
            >
              Password
            </label>
```

```
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              style={{
                width: "100%",
                padding: "10px",
                border: "1px solid #ddd",
                borderRadius: "5px",
                boxSizing: "border-box",
              }}
              required
            />
          </div>

          <button
            type="submit"
            style={{
              width: "100%",
              padding: "12px",
              backgroundColor: "#007bff",
              color: "white",
              border: "none",
              borderRadius: "5px",
              cursor: "pointer",
              fontSize: "16px",
            }}
          >
            Login
          </button>
        </form>

        <div
          style={{
            marginTop: "30px",
            padding: "15px",
            backgroundColor: "#f8f9fa",
            borderRadius: "5px",
          }}
        >
          <h4 style={{ margin: "0 0 10px 0" }}>Demo Credentials:</h4>

          <p style={{ margin: "5px 0", fontSize: "14px" }}>
            <strong>Admin:</strong> admin_24bai71005@cuchd.in / admin_24BAI71005
          </p>

          <p style={{ margin: "5px 0", fontSize: "14px" }}>
            <strong>Editor:</strong> editor_24bai71005@cuchd.in / editor_24BAI71005
          </p>

          <p style={{ margin: "5px 0", fontSize: "14px" }}>
            <strong>Viewer:</strong> viewer_24bai71005@cuchd.in / viewer_24BAI71005
          </p>
        </div>
      </div>
```

```
    </div>
  );
}

export default Login;
```

Admin . js :

```
import Navbar from "../components/Navbar";

function Admin() {
  return (
    <div>
      <Navbar />

      <div style={{ padding: "20px" }}>
        <h1>Admin Dashboard</h1>

        <div
          style={{
            display: "grid",
            gridTemplateColumns: "repeat(auto-fit, minmax(250px, 1fr))",
            gap: "20px",
            marginTop: "30px",
          }}
        >
          {/* User Management Card */}
          <div style={cardStyle}>
            <h3>👥 User Management</h3>
            <p>Manage all users, create new accounts, and assign roles</p>
            <button style={buttonStyle}>Manage Users</button>
          </div>

          {/* Reports Card */}
          <div style={cardStyle}>
            <h3>📊 Reports</h3>
            <p>View all system reports and analytics</p>
            <button style={buttonStyle}>View Reports</button>
          </div>

          {/* Settings Card */}
          <div style={cardStyle}>
            <h3>⚙️ Settings</h3>
            <p>Configure system settings and preferences</p>
            <button style={buttonStyle}>Open Settings</button>
          </div>

          {/* Admin Actions */}
          <div style={cardStyle}>
            <h3>🔒 Admin Actions</h3>
            <p>Access administrative functions and system logs</p>
            <button style={buttonStyle}>Admin Panel</button>
          </div>
        </div>

        <div
          style={{
            marginTop: "40px",
```

```
            padding: "20px",
            backgroundColor: "#e7f3ff",
            borderRadius: "10px",
          }}
        >
          <h3>Admin Privileges:</h3>

          <ul>
            <li>Full system access</li>
            <li>User management capabilities</li>
            <li>System configuration</li>
            <li>View all reports</li>
            <li>Manage content</li>
          </ul>
        </div>
      </div>
    </div>
  );
}

const cardStyle = {
  backgroundColor: "white",
  padding: "20px",
  borderRadius: "10px",
  boxShadow: "0 2px 5px rgba(0,0,0,0.1)",
};

const buttonStyle = {
  padding: "10px 20px",
  backgroundColor: "#007bff",
  color: "white",
  border: "none",
  borderRadius: "5px",
  cursor: "pointer",
  marginTop: "10px",
};

export default Admin;
```

Editor . js :

```
import Navbar from "../components/Navbar";

function Editor() {
  return (
    <div>
      <Navbar />

      <div style={{ padding: "20px" }}>
        <h1>Editor Dashboard</h1>

        <div
          style={{
            display: "grid",
            gridTemplateColumns: "repeat(auto-fit, minmax(250px, 1fr))",
            gap: "20px",
            marginTop: "30px",
          }}
```

```
        >
          {/* Manage Content Card */}
          <div style={cardStyle}>
            <h3>📝 Manage Content</h3>
            <p>Create, edit, and manage website content</p>
            <button style={buttonStyle}>Manage Content</button>
          </div>

          {/* Edit Articles Card */}
          <div style={cardStyle}>
            <h3>🖍️ Edit Articles</h3>
            <p>Write and edit articles for publication</p>
            <button style={buttonStyle}>Edit Articles</button>
          </div>

          {/* View Reports Card */}
          <div style={cardStyle}>
            <h3>📈 View Reports</h3>
            <p>Access content performance reports</p>
            <button style={buttonStyle}>View Reports</button>
          </div>
        </div>

        <div
          style={{
            marginTop: "40px",
            padding: "20px",
            backgroundColor: "#e7f3ff",
            borderRadius: "10px",
          }}
        >
          <h3>Editor Privileges:</h3>

          <ul>
            <li>Manage website content</li>
            <li>Edit and publish articles</li>
            <li>View content reports</li>
            <li>Cannot access user management</li>
            <li>Cannot modify system settings</li>
          </ul>
        </div>
      </div>
    </div>
  );
}

const cardStyle = {
  backgroundColor: "white",
  padding: "20px",
  borderRadius: "10px",
  boxShadow: "0 2px 5px rgba(0,0,0,0.1)",
};

const buttonStyle = {
  padding: "10px 20px",
  backgroundColor: "#28a745",
```

```
  color: "white",
  border: "none",
  borderRadius: "5px",
  cursor: "pointer",
  marginTop: "10px",
};

export default Editor;
```

Viewer . js :

```
import Navbar from "../components/Navbar";

function Viewer() {
  return (
    <div>
      <Navbar />

      <div style={{ padding: "20px" }}>
        <h1>Viewer Dashboard</h1>

        <div
          style={{
            display: "grid",
            gridTemplateColumns: "repeat(auto-fit, minmax(250px, 1fr))",
            gap: "20px",
            marginTop: "30px",
          }}
        >
          {/* View Content Card */}
          <div style={cardStyle}>
            <h3>👁️ View Content</h3>
            <p>Browse and read available content</p>
            <button style={buttonStyle}>View Content</button>
          </div>

          {/* View Reports Card */}
          <div style={cardStyle}>
            <h3>📊 View Reports</h3>
            <p>Access read-only reports</p>
            <button style={buttonStyle}>View Reports</button>
          </div>

          {/* Profile Card */}
          <div style={cardStyle}>
            <h3>👤 Profile</h3>
            <p>View and update your profile information</p>
            <button style={buttonStyle}>My Profile</button>
          </div>
        </div>

        <div
          style={{
            marginTop: "40px",
            padding: "20px",
            backgroundColor: "#e7f3ff",
            borderRadius: "10px",
          }}
```

```
        >
          <h3>Viewer Privileges:</h3>

          <ul>
            <li>Read-only access to content</li>
            <li>View reports</li>
            <li>Manage personal profile</li>
            <li>Cannot edit or create content</li>
            <li>No administrative access</li>
          </ul>
        </div>
      </div>
    </div>
  );
}

const cardStyle = {
  backgroundColor: "white",
  padding: "20px",
  borderRadius: "10px",
  boxShadow: "0 2px 5px rgba(0,0,0,0.1)",
};

const buttonStyle = {
  padding: "10px 20px",
  backgroundColor: "#6c757d",
  color: "white",
  border: "none",
  borderRadius: "5px",
  cursor: "pointer",
  marginTop: "10px",
};

export default Viewer;
```

App . js :

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Login from "./pages/Login";
import Admin from "./pages/Admin";
import Editor from "./pages/Editor";
import Viewer from "./pages/Viewer";
import Unauthorized from "./components/Unauthorized";
import ProtectedRoute from "./components/ProtectedRoute";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />

        <Route
          path="/admin"
          element={
            <ProtectedRoute allowedRoles={["Admin"]}>
              <Admin />
            </ProtectedRoute>
          }
```

```
        />

        <Route
          path="/editor"
          element={
            <ProtectedRoute allowedRoles={["Editor"]}>
              <Editor />
            </ProtectedRoute>
          }
        />

        <Route
          path="/viewer"
          element={
            <ProtectedRoute allowedRoles={["Viewer"]}>
              <Viewer />
            </ProtectedRoute>
          }
        />

        <Route
          path="/unauthorized"
          element={<Unauthorized />}
        />

        {/* Catch all undefined routes */}
        <Route
          path="*"
          element={<Unauthorized />}
        />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

7. OUTPUT :

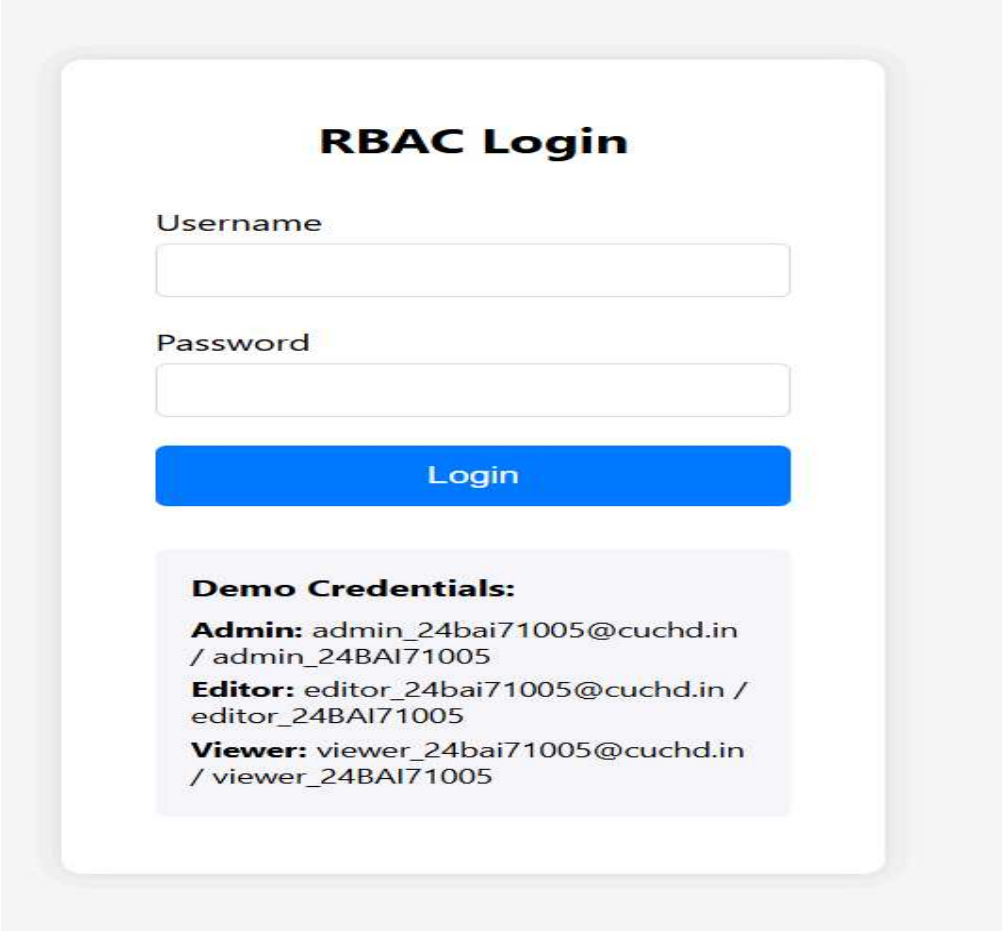


Fig 3.2: RBAC Login Page

This experiment demonstrates the implementation of **Role-Based Access Control (RBAC)** using **React.js** with a simple **JWT-like authentication mechanism**. Users log in with predefined credentials, and a token containing their username and role is generated and stored in the browser's local storage. Based on the assigned role (**Admin**, **Editor**, or **Viewer**), the application redirects users to their respective dashboards and restricts access to unauthorized pages using protected routes. This ensures that users can only access the resources and functionalities permitted for their roles, providing secure and organized access control within the application.

8. LEARNING OUTCOME :

- Understand the concept of authorization and Role-Based Access Control (RBAC).
- Implement protected routes using React Router.
- Restrict application access based on user roles.
- Dynamically render user interfaces according to permissions.
- Manage user roles and secure navigation in React applications.
- Implement role-based authentication and authorization for enhanced application security.
- Develop secure, scalable, and permission-based web applications.